

AI Chapter 04

★ Best First Search (বেস্ট ফার্স্ট সার্চ)

Best First Search হলো একটি **Search Algorithm** (অনুসন্ধান অ্যালগরিদম) যা সব সময় সেই **Node** (নোড) টা আগে নির্বাচন করে যেটা দেখে মনে হয় **Goal** (লক্ষ্য) এর সবচেয়ে কাছে পৌঁছাবে।

👉 সহজভাবে:

Choose node which appears best (যে নোড সবচেয়ে ভালো মনে হয় সেটাকেই আগে বেছে নেয়)।

✅ Evaluation Function — $f(n)$

Best First Search এ প্রতিটি Node কে একটি মান দিয়ে বিচার করা হয়।

এটাকে বলে —

👉 **Evaluation Function — $f(n)$**

Definition:

Node is selected for expansion based on an evaluation function $f(n)$

বাংলা ব্যাখ্যা:

একটি Node expand (পরবর্তী ধাপে নেওয়া) হবে কিনা তা নির্ভর করে তার **$f(n)$** মানের উপর।

👉 $f(n)$ দেখে algorithm বুঝে:

- কোন Node ভালো?
 - কোনটা আগে explore করা উচিত?
-

★ Evaluation Function কী করে?

এটা মূলত আন্দাজ করে —

Goal পর্যন্ত যেতে কত দূর বাকি আছে।

মানে —

👉 Goal এর কাছে যেটা বেশি মনে হয় সেটাকে আগে নেয়।

★ Heuristic Function — $h(n)$

Best First Search এ সবচেয়ে গুরুত্বপূর্ণ জিনিস হলো —

👉 **Heuristic Function** (হিউরিস্টিক ফাংশন)।

Definition:

$h(n)$ = estimated cost of the cheapest path from node n to goal node.

বাংলা ব্যাখ্যা:

$h(n)$ মানে হলো —

👉 একটি Node থেকে Goal পর্যন্ত যেতে কত খরচ বা কত দূর লাগতে পারে — তার একটি অনুমান (estimate)।

⚠️ মনে রাখবে:

এটা exact না।

এটা শুধু আন্দাজ।

★ Heuristic মানে কী?

Dictionary অনুযায়ী:

“A rule of thumb, simplification, or educated guess.”

মানে —

👉 একটি Practical Shortcut বা সহজ বুদ্ধি।

★ Heuristic এর ৩টা গুরুত্বপূর্ণ ধারণা

✓ 1. Rule of Thumb (সহজ অভিজ্ঞতার নিয়ম)

এটা কোনো কঠিন formula না।

এটা সাধারণ অভিজ্ঞতা থেকে পাওয়া নিয়ম।

Example:

তুমি জঙ্গলে হারিয়ে গেলে —

👉 নদীর পাশে হাঁটো।

কারণ সাধারণত নদীর পাশে মানুষ থাকে।

এটা guarantee না।

কিন্তু বেশিরভাগ সময় কাজ করে।

✓ 2. Simplification (সমস্যা সহজ করা)

Complex problem কে সহজ করে ফেলা।

Example:

Market এ গেলে —

কম ingredient আছে এমন খাবার নিলে healthy হবে।

তুমি সব analyse করছ না।

Short decision নিচ্ছ।

✓ 3. Educated Guess (জ্ঞানভিত্তিক অনুমান)

অভিজ্ঞতা দিয়ে আন্দাজ করা।

Example:

Computer slow হলে —

Restart দিলে ঠিক হতে পারে।

তুমি exact জানো না।

কিন্তু experience বলে এটা কাজ করে।

★ Heuristic কেন দরকার?

সব option check করলে অনেক সময় লাগে।

তাই heuristic সাহায্য করে —

Likely solution এর দিকে focus করতে।

Example:

তোমার কাছে ২০টা চাবি আছে।

সব try না করে —

👉 যেটা বেশি ব্যবহার করা মনে হয় আগে সেটাই চেষ্টা করো।

এটাই heuristic।

★ Domains that are difficult and poorly understood

কিছু problem আছে যেগুলোর exact formula নাই।

যেমন:

Stock Market।

আগাম price predict করা কঠিন।

তাই মানুষ heuristic ব্যবহার করে।

Example:

👉 Buy low, sell high.

এটা exact science না।

কিন্তু guideline।

★ Best First Search Implementation

Implementation এ একটা data structure লাগে।

✓ Fringe (Open List)

এটা হলো waiting Node list।

যেগুলো explore করা হবে।

★ Priority Queue ব্যবহার হয়

Fringe থাকে —

👉 **Priority Queue** (অগ্রাধিকার তালিকা)।

এখানে Node গুলো সাজানো থাকে —

👉 $f(n)$ ছোট থেকে বড়।

মানে —

যার heuristic কম (goal এর কাছে মনে হয়) —

সেটা আগে আসবে।

Example:

ধরি:

Node A $\rightarrow h(n)=10$

Node B $\rightarrow h(n)=3$

Node C $\rightarrow h(n)=7$

Priority Queue হবে:

$B \rightarrow C \rightarrow A$

কারণ:

B goal এর সবচেয়ে কাছে মনে হচ্ছে।

★ এক লাইনে Best First Search

👉 Goal এর দিকে দ্রুত যেতে heuristic ব্যবহার করে যে node সবচেয়ে promising মনে হয় সেটাকে আগে explore করে।

★ একদম সহজ Real Life Example

তুমি ঢাকা থেকে চট্টগ্রাম যেতে চাও।

Google Map দেখলে:

Route 1 → 300 km

Route 2 → 200 km

Route 3 → 400 km

তুমি 200km টা আগে নেবে।

কারণ মনে হচ্ছে দ্রুত পৌঁছাবে।

এটাই Best First Search।

★ Heuristic Final Definition

👉 A heuristic is a quick and practical way to solve problems when:

- Time কম
- Information কম
- Problem complex

এটা perfect না।

কিন্তু বেশিরভাগ সময় ভালো কাজ করে।

ঠিক আছে 🍌 এবার আমি **Greedy Best First Search (GBFS)** টা একদম **clean + concept clear + exam ready style** এ বাংলায় (বাংলা অক্ষর ব্যবহার করে) বুঝাইতেছি, সাথে English term রাখতেছি যাতে পড়লেই বুঝে।

★ Greedy Best First Search (GBFS)

Greedy Best First Search হলো এমন একটি **Search Algorithm** (অনুসন্ধান অ্যালগরিদম) যা সব সময় সেই Node নির্বাচন করে যেটা **Goal** এর সবচেয়ে কাছাকাছি মনে হয়।

🍌 এখানে শুধু একটা জিনিস দেখে:

Heuristic Function — $h(n)$

✓ Evaluation Function

Greedy Best First Search এ —

$$f(n) = h(n)$$

মানে:

🍌 শুধুমাত্র heuristic value দেখে decision নেয়।

$h(n)$ কী?

$h(n)$ = estimated distance from node n to goal.

বাংলায়:

একটি Node থেকে Goal পর্যন্ত যেতে কত দূর লাগতে পারে তার অনুমান।

⚠ Important:

এখানে **Past Cost ($g(n)$)** ধরা হয় না।

মানে:

- কত দূর already আসছে → ignore।
 - সামনে কত কাছে মনে হচ্ছে → শুধু সেটাই দেখে।
-

★ Greedy মানে কী?

Greedy মানে:

👉 এখন সবচেয়ে লাভজনক মনে হচ্ছে যেটা সেটাই নেওয়া।

Future চিন্তা করে না।

Example:

Maze (গোলকধাঁধা) এ আছে।

তুমি শুধু সামনে shortest মনে হচ্ছে এমন রাস্তা ধরছো।

Dead End আছে কিনা দেখছো না।

ফলাফল:

👉 আটকে যেতে পারো।

★ Complete বা Optimal?

✗ Complete — NO

Complete মানে:

সবসময় কি Goal খুঁজে পাবে?

উত্তর:

না।

কেন Not Complete?

কারণ GBFS Loop এ আটকে যেতে পারে।

একই জায়গায় বারবার যেতে পারে।

Goal miss করতে পারে।

Example:

$A \rightarrow B \rightarrow C \rightarrow B \rightarrow C \rightarrow B$

এভাবে ঘুরতেই থাকবে।

Goal এ পৌঁছাবে না।

✗ Optimal — NO

Optimal মানে:

সবচেয়ে কম cost এর solution কি দিবে?

উত্তর:

না।

কেন?

কারণ শুধু $h(n)$ দেখে।

Total cost দেখে না।

Difference বুঝো:

$g(n)$ = already কত খরচ হয়েছে।

$h(n)$ = সামনে কত লাগতে পারে।

GBFS শুধু $h(n)$ দেখে।

Example:

Path 1:

Already travelled = 500 km

Goal distance = 10 km

Path 2:

Already travelled = 20 km

Goal distance = 50 km

GBFS নেবে:

👉 Path 1।

কারণ $h(n)=10$ ছোট।

কিন্তু total cost অনেক বেশি।

★ False Start Problem

Slide এ বলা হয়েছে:

Minimizing $h(n)$ can result in false starts.

False Start মানে?

ভুল পথে শুরু করা।

দেখতে ভালো লাগছে।

কিন্তু পরে সমস্যা।

★ Example — Iasi → Fagaras

ধরি Romania Map problem।

GBFS দেখে:

Goal এর কাছাকাছি কোন শহর?

$h(n)$ কম যেটা।

সেটাই নেয়।

কিন্তু সেই রাস্তা হয়তো অনেক ঘুরে গেছে।

অন্য রাস্তা ছোট ছিল।

কারণ:

👉 GBFS past cost জানে না।

★ কে এই সমস্যা এড়ায়?

👉 A* Search।

কারণ:

A* ব্যবহার করে:

$$f(n) = g(n) + h(n)$$

মানে:

- কত দূর আসছো।
- সামনে কত দূর।

দুটাই দেখে।

★ Check on Repeated States

Slide এ বলা হয়েছে:

Check on repeated states.

মানে কী?

একই Node বারবার explore করা যাবে না।

কেন?

না হলে:

Loop হবে।

সময় নষ্ট হবে।

Memory নষ্ট হবে।

Example:

Dhaka → Gazipur → Dhaka → Gazipur

এভাবে ঘুরতেই থাকবে।

Solution:

Visited List রাখতে হয়।

আগে visited হলে skip।

★ Time and Space Complexity

Worst Case এ —

সব Node generate হতে পারে।

Complexity:

$$O(b^m)$$

এখানে:

b = Branching Factor

মানে:

একটা Node থেকে কয়টা child বের হয়।

Example:

৪টা রাস্তা থাকলে:

$$b=4।$$

m = Maximum Depth

মানে:

Search tree কত গভীর।

Worst Case:

সব level explore করলে।

সময় বেশি লাগে।

Memory ও বেশি লাগে।

Good Heuristic কেন গুরুত্বপূর্ণ?

Slide এ বলা হয়েছে:

Good heuristic can give dramatic improvement.

মানে:

ভালো আন্দাজ দিলে।

কম Node explore হবে।

দ্রুত Goal পাওয়া যাবে।

Example:

Google Map।

ঠিক traffic data থাকলে দ্রুত route দেয়।

ভুল data হলে ঘুরাবে।

Summary (Exam Ready)

Greedy Best First Search uses only heuristic function $h(n)$ to expand the node that appears closest to the goal. It is neither complete nor optimal because it ignores past path cost $g(n)$. Minimizing $h(n)$ may cause false starts and loops. Checking repeated states helps avoid infinite loops. Worst case complexity is $O(b^m)$, but a good heuristic significantly improves performance.

ঠিক আছে 👍 এবার *A Search* (এ-স্টার সার্চ)* একদম **deep concept** + সহজ বাংলা + **English term** সহ + **exam ready** করে বুঝাইতেছি যাতে পড়েই বুঝতে পারো।

★ A* Search (A-Star Search)

A* হলো —

Best-known form of Best First Search.

মানে:

Best First Search এর সবচেয়ে জনপ্রিয় এবং শক্তিশালী version।

★ মূল Idea (Main Idea)

Slide এ বলা হয়েছে:

Avoid expanding paths that are already expensive.

বাংলা ব্যাখ্যা:

যে পথ ইতিমধ্যে অনেক বেশি খরচ হয়ে গেছে —

👉 সেই পথ আর explore করবে না।

মানে:

Algorithm smart ভাবে কাজ করে।

শুধু সামনে কাছে মনে হচ্ছে সেটা না।

পেছনে কত খরচ হয়েছে সেটাও দেখে।

★ Evaluation Function

A* এর সবচেয়ে গুরুত্বপূর্ণ formula:

$$✓ f(n) = g(n) + h(n)$$

✓ g(n) কী?

Cost so far.

বাংলায়:

Start Node থেকে বর্তমান Node পর্যন্ত আসতে কত cost হয়েছে।

Example:

Dhaka → Comilla → Feni।

তুমি already 200 km এসেছো।

তাহলে:

$$g(n)=200।$$

✓ h(n) কী?

Estimated cost to reach Goal.

বাংলায়:

এখন থেকে Goal পর্যন্ত যেতে কত লাগতে পারে — তার অনুমান।

Example:

Feni → Chittagong।

আরও 80 km লাগতে পারে।

তাহলে:

$$h(n)=80।$$

✓ $f(n)$ কী?

Estimated total cost.

মান্নে:

পুরো journey শেষ করতে মোট কত লাগতে পারে।

Example:

$$g(n)=200$$

$$h(n)=80$$

তাহলে:

$$\text{👉 } f(n)=280।$$

A* সবসময় কম $f(n)$ যেটা সেটাকে আগে নেয়।

★ A* কেন ভালো?

কারণ:

GBFS শুধু সামনে দেখে।

UCS শুধু পেছনের cost দেখে।

কিন্তু A* দেখে:

👉 Past + Future দুটোই।

★ Complete এবং Optimal?

Slide এ বলা হয়েছে:

A* search is both complete and optimal if $h(n)$ satisfies certain conditions.

✓ Complete

Complete মানে:

Goal থাকলে অবশ্যই খুঁজে পাবে।

A* সাধারণত Complete।

Optimal

Optimal মানে:

সবচেয়ে কম cost এর solution দিবে।

কিন্তু একটা শর্ত আছে।

Admissible Heuristic

Slide এ বলা হয়েছে:

A* search is optimal if $h(n)$ is admissible.

Admissible মানে কী?

Heuristic কখনো Goal distance বেশি দেখাবে না।

Definition:

$$h(n) \leq h^*(n)$$

এখানে:

$h(n)$ = heuristic estimate।

$h^*(n)$ = actual true cost।

Example:

Actual distance:

100 km।

Heuristic বলে:

80 km।

OK।

কারণ underestimate করছে।

কিন্তু যদি বলে:

150 km।

তাহলে overestimate।

এটা admissible না।

★ কেন **admissible** দরকার?

কারণ:

Algorithm ভুল পথে যেতে পারে।

যদি heuristic বেশি exaggerate করে।

★ **Optimistic Heuristic**

Slide এ বলা হয়েছে:

Admissible heuristics are optimistic.

মানে:

সবসময় ভাবে —

Goal আসলে আরও কাছে।

Example:

Straight Line Distance।

বাস্তবে road বাঁকা।

তাই straight distance সবসময় কম হয়।

Example:

Google Map এ bird view।

★ hSLD(n)

Straight Line Distance heuristic।

Road distance কখনো এর কম না।

তাই overestimate করে না।

এটা admissible।

★ OPTIMALITY OF A*

এটা একটু concept heavy।

আমি একদম সহজ করে explain করছি।

Situation:

ধরি দুইটা Goal আছে।

G1 → optimal goal।

G2 → suboptimal goal।

Optimal Solution Cost:

👉 C^*

মানে:

সবচেয়ে কম cost।

ধরি:

G2 fringe এ আছে।

মানে explore করার অপেক্ষায়।

Goal Node এর জন্য:

সবসময়:

$$h(\text{goal})=0।$$

কারণ Goal এ পৌঁছে গেছো।

আর distance নাই।

তাই:

$$f(G2)=g(G2)+0।$$

মানে:

$$f(G2)=g(G2)।$$

ধরি:

G2 suboptimal।

মানে:

$$g(G2) > C^*$$

অর্থাৎ:

Optimal solution এর চেয়ে বেশি cost।

এখন **Optimal Path** এর একটা **Node** নাও:

ধরি Node n ।

যেটা optimal path এ আছে।

Admissible heuristic হলে:

$h(n)$ কখনো বেশি দেখাবে না।

তাই:

$$f(n) = g(n) + h(n) \leq C^*$$

মানে:

$f(n)$ optimal solution cost এর চেয়ে বড় হবে না।

এখন compare করো:

$$f(n) \leq C^*$$

আর:

$$f(G2) > C^*$$

মানে:

$$f(n) < f(G2)$$

A^* সবসময় ছোট $f(n)$ আগে expand করে।

তাই:

👉 Optimal path এর Node আগে expand হবে।

$G2$ expand হবে না।

শেষে:

A* optimal goal return করবে।

★ Simple Story Example

তুমি restaurant খুঁজছো।

Route A:

Already walked = 10 min

Remaining guess = 5 min

$f=15$ ।

Route B:

Already walked = 2 min

Remaining guess = 20 min।

$f=22$ ।

A* নেবে Route A।

কারণ total estimate কম।

★ Time Complexity

Worst case এ:

অনেক Node explore করতে পারে।

Memory বেশি লাগে।

কিন্তু:

Good heuristic হলে fast।

★ Summary (Exam Ready)

A* search is the most effective best-first search algorithm that evaluates nodes using $f(n)=g(n)+h(n)$. It considers both the cost already incurred and the estimated cost to reach the goal. A* is complete and optimal if the heuristic function is admissible, meaning it never overestimates the true cost. By expanding nodes with the lowest estimated total cost, A* guarantees an optimal solution.

ঠিক আছে 👍 এবার A Search Evaluation (Complete, Optimal, Efficiency, Time & Space Complexity)* একদম **clear concept** + বাংলা অক্ষরে **explanation** + **English term** সহ + **exam ready style** এ বুঝাইতেছি।

★ A* Search — Evaluation

A* Search একটি খুব শক্তিশালী **Informed Search Algorithm** (জ্ঞানভিত্তিক অনুসন্ধান অ্যালগরিদম) কারণ এটি একই সাথে —

- ✓ Past Cost দেখে
- ✓ Future Estimate দেখে।

Formula:

$$f(n) = g(n) + h(n)$$

★ Complete (সম্পূর্ণতা)

Slide এ বলা হয়েছে:

Complete: Yes
Unless there are infinitely many nodes with $f < f(G)$

✓ Complete মানে কী?

Complete মানে:

👉 যদি Goal থাকে তাহলে Algorithm অবশ্যই Goal খুঁজে পাবে।

কেন A* Complete?

কারণ:

A* সবসময় ছোট $f(n)$ আগে expand করে।

Goal পাওয়া পর্যন্ত explore করে।

Exception (ব্যতিক্রম)

যদি এমন হয় —

Goal এর আগে অসংখ্য Node আছে যাদের:

$$f(n) < f(G)$$

মানে:

Goal এর estimated cost থেকেও কম।

আর সেই Node গুলো infinite।

তাহলে A* কখনো Goal expand করার সুযোগ পাবে না।

Example:

ধরি:

Goal এর $f(G)=50$ ।

কিন্তু infinite Node আছে যাদের:

$f=10,20,30,40$ ।

Algorithm এগুলোই explore করতে থাকবে।

Goal এ পৌঁছাবে না।

তাই বলা হয়েছে:

👉 Infinite cheaper node থাকলে Complete guarantee না।

কিন্তু practical problem এ এটা খুব rare।

★ Optimal (সর্বোত্তম)

Slide:

Optimal: Yes.

Optimal মানে কী?

সবচেয়ে কম cost এর solution দিবে।

কেন A* Optimal?

যদি heuristic হয় —

👉 **Admissible heuristic**।

মানে:

Goal distance কখনো বেশি দেখাবে না।

কারণ:

A* expand করে smallest $f(n)$ ।

Optimal path এর Node গুলো আগে expand হয়।

Suboptimal Goal ignore হয়।

★ Optimally Efficient

Slide এ বলা হয়েছে:

A* is also optimally efficient for any given $h(n)$.

এর মানে কী?

যদি একই heuristic $h(n)$ ব্যবহার করো —

👉 কোনো optimal algorithm নিশ্চিতভাবে A^* এর চেয়ে কম Node expand করতে পারবে না।

সহজ ভাষায়:

A^* যত কম কাজ করে optimal solution বের করে —

তার চেয়ে কম কাজ করে কেউ guarantee দিতে পারে না।

Example:

ধরি:

একটা maze আছে।

১০০০টা Node explore করা লাগবে।

A^* ২০০ Node explore করেই solution পেল।

অন্য algorithm হয়তো ৩০০ explore করবে।

কিন্তু ২০০ এর কম explore করে optimal solution guarantee করা সম্ভব না।

মানে:

👉 A^* already smartest।

★ Time Complexity

Slide:

Number of nodes expanded is still exponential.

Time Complexity:

Worst Case:

Exponential।

মানে:

Solution depth যত বাড়বে।

Node সংখ্যা দ্রুত বাড়বে।

Concept:

$$O(b^d)$$

এখানে:

b = branching factor।

মানে:

এক Node থেকে কয়টা child বের হয়।

d = solution depth।

Goal কত দূরে।

Example:

প্রতি Node এ ৩টা রাস্তা।

Depth = 10।

Total Node প্রায়:

$$3^{10}$$

অনেক বড় সংখ্যা।

তাই:

Large problem এ সময় বেশি লাগে।

কিন্তু **Good Heuristic** হলে?

অনেক Node skip করে।

দ্রুত solution।

★ **Space Complexity** (সবচেয়ে বড় সমস্যা)

Slide:

All generated nodes are kept in memory.

মানে কী?

A* Open list + Closed list রাখে।

সব Node memory তে জমা রাখে।

Example:

১০ লাখ Node generate করলে।

সব RAM এ থাকবে।

Memory দ্রুত শেষ হয়ে যায়।

★ **Biggest Problem of A***

Slide:

A* usually runs out of space before running out of time.

মানে:

সময় শেষ হওয়ার আগে RAM শেষ হয়ে যায়।

Real Example:

Google Map huge city search করছে।

Node লাখ লাখ।

CPU সময় আছে।

কিন্তু Memory full।

Program crash।

তাই:

A* এর biggest limitation:

👉 Memory consumption।

★ Summary (Exam Ready Answer)

A* search is complete unless infinitely many nodes exist with f-values less than the goal node. It is optimal when the heuristic function is admissible. A* is optimally efficient because no other optimal algorithm using the same heuristic expands fewer nodes. However, its time complexity is exponential in the worst case, and its space complexity is high because all generated nodes must be stored in memory. Therefore, A* often runs out of memory before time.

🔥 চাইলে আমি তোমাকে একদম “A Optimality Proof Short Trick (Exam এ ১০০% আসলে ১ মিনিটে লিখার version)”* শিখাই দিতে পারি 😊🔥। বললেই দিব 👍।

★ Search Algorithms — Time & Space Complexity Chart

Algorithm	Evaluation Idea	Time Complexity (Worst Case)	Space Complexity	Important Notes
Best First Search	$f(n)$ দিয়ে best node নেয়	$O(b^m)$	$O(b^m)$	heuristic এর উপর অনেক depend করে
Greedy Best First Search (GBFS)	শুধু $h(n)$ minimize করে	$O(b^m)$	$O(b^m)$	fast কিন্তু optimal না
A Search*	$f(n) = g(n) + h(n)$	$O(b^d)$ (exponential)	$O(b^d)$	memory সবচেয়ে বেশি লাগে
Hill Climbing	local neighbour improve করে	$O(b \times m)$ (approx)	$O(1)$	শুধু current node রাখে
Local Beam Search	k best state একসাথে রাখে	$O(k \times b \times m)$	$O(k)$	multiple path explore করে

ঠিক আছে 👍 এবার আমি **Local Search Algorithms and Optimization Problems** একদম **clean** + সুন্দর + বাংলা অক্ষরে **explanation + English term** সহ এমনভাবে বুঝাইতেছি যাতে পড়লেই concept clear হয়ে যায়।

★ Local Search Algorithms and Optimization Problems

আগে আমরা যেসব Search Algorithm পড়েছি (যেমন BFS, A*, Best First Search) —

সেগুলো কী করতো?

👉 পুরো Search Space systematic ভাবে explore করতো।

মানে:

Start থেকে Goal পর্যন্ত পুরো **Path** (পথ) খুঁজতো।

★ Previous Search Algorithms

আগের Algorithm গুলোর লক্ষ্য ছিল:

Path to goal is solution.

মানে:

Goal এ যাওয়ার পুরো রাস্তা important।

Example:

Google Map।

Dhaka থেকে Chittagong যেতে হলে:

- কোন রাস্তা যাবে?
- কোথায় turn নেবে?

সব দরকার।

কারণ Path গুরুত্বপূর্ণ।

★ কিন্তু কিছু Problem এ Path গুরুত্বপূর্ণ না

Slide এ বলা হয়েছে:

For some problems, it is sufficient to find a solution.

মানে:

Goal state পেলেই হলো।

কিভাবে পৌঁছালে সেটা matter না।

★ Example — 8 Queens Problem

(Chess board problem)

৮টা Queen বসাতে হবে যেন কেউ কাউকে attack না করে।

এখানে:

👉 Queen কোথা দিয়ে move করে বসালে সেটা দরকার নাই।

শুধু Final Board configuration দরকার।

মানে:

Path irrelevant।

★ তখন **Local Search** ব্যবহার হয়

★ **Local Search** কী?

Definition:

Local search uses a single current state and moves to neighboring states.

বাংলা ব্যাখ্যা:

একটা Current State থাকে।

সেখান থেকে আশেপাশের (Neighbor) state গুলো দেখে।

যেটা ভালো লাগে সেখানে move করে।

Example:

ভূমি পাহাড়ে উঠছিো।

চারদিকে তাকাও।

যেদিকে উঁচু মনে হয় সেখানে যাও।

পুরো map দেখছিো না।

★ **Memory Requirement**

Slide:

Memory requirements can be dramatically relaxed.

মানে:

অনেক কম Memory লাগে।

কেন?

কারণ:

পুরোনো state রেখে দেয় না।

Slide:

All previous states can be discarded.

মানে:

আগের state delete।

শুধু বর্তমানটা থাকে।

Example:

তুমি হাঁটছো।

প্রতি পদক্ষেপের history লিখে রাখছো না।

শুধু এখন কোথায় আছো জানো।

তাই:

👉 Memory খুব কম লাগে।

★ কেন **Local** বলা হয়?

Slide:

Only information about the current state is kept.

মানে:

শুধু current state জানে।

পুরো search tree না।

তাই নাম:

👉 Local Search।

★ Advantages of Local Search

✓ 1. Very Little Memory

কারণ:

একটা state রাখে।

Space Complexity কম।

✓ 2. Large বা Infinite Space এ কাজ করে

Huge problem এ কাজ করে।

Example:

Millions configuration।

Tree search impossible।

Local search পারে।

✓ 3. Optimization Problem এ Useful

Slide:

Useful for pure optimization problems.

Optimization মানে:

Best solution খোঁজা।

Example:

- Best schedule।
 - Best route।
 - Best investment।
-

★ Objective Function

Local Search একটা function ব্যবহার করে।

এটাকে বলে:

Objective Function।

এটা বলে দেয়:

কোন state ভালো।

Example:

8 queens।

Conflict যত কম।

তত ভালো।

AI চেষ্টা করে:

👉 Conflict = 0।

★ Survival of the Fittest (Metaphor)

Slide:

Survival of the fittest.

মানে:

Nature এর মতো।

যে strongest সে survive করে।

Algorithm এ:

Better state survive করে।

Bad state বাদ পড়ে।

Example:

Evolution।

Strong animal বেঁচে যায়।

Weak eliminate।

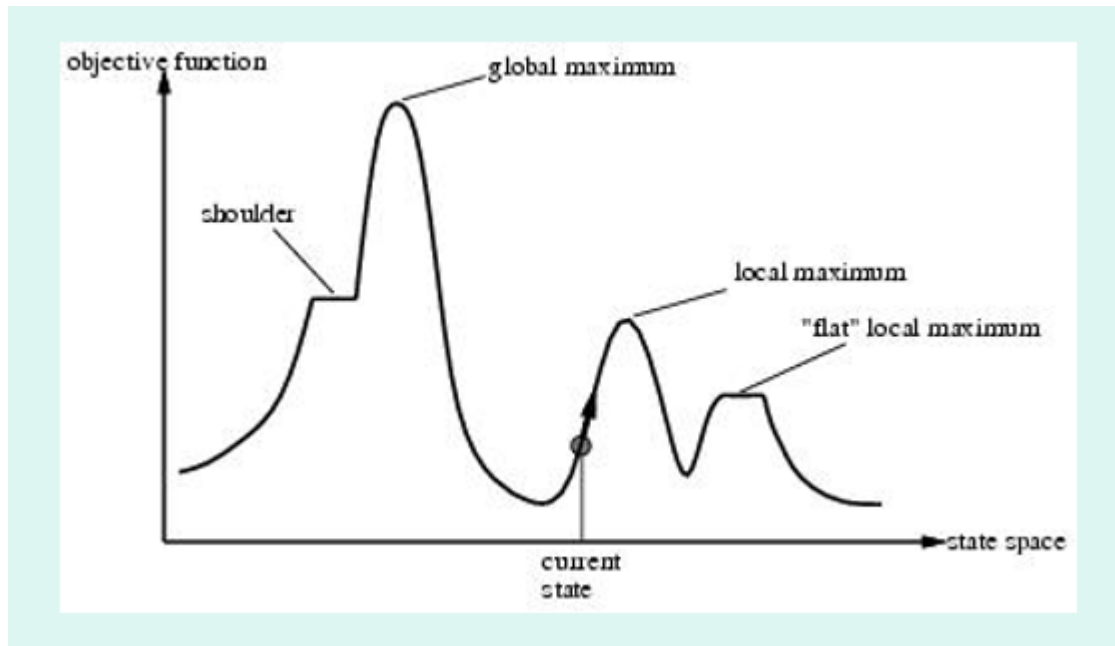
Local Search এ:

Better solution → থাকে।

Bad solution → বাদ।

★ Summary (Exam Ready Answer)

Local search algorithms operate by maintaining a single current state and moving to neighboring states to improve solution quality. Unlike systematic search methods, the path to the goal is not important; only the final state matters. These algorithms require very little memory because previous states are discarded. Local search is highly effective for large or infinite search spaces and optimization problems where the objective is to find the best possible solution according to an objective function.



ঠিক আছে 👍 এবার **Hill Climbing Search** (হিল ক্লাইম্বিং সার্চ) একদম **concept clear + বাংলা অঙ্করে explanation + English term সহ + example** ভিত্তিক সুন্দরভাবে বুঝাইতেছি যাতে পড়লেই পুরোটা মাথায় বসে যায়।

★ Hill Climbing Search (হিল ক্লাইম্বিং সার্চ)

Hill Climbing হলো একটি **Local Search Algorithm** (লোকাল সার্চ অ্যালগরিদম)।

এটা সবসময় চেষ্টা করে —

Better state এর দিকে **move** করতে।

★ Main Idea

Slide:

Continually moves uphill (increasing value of evaluation function).

বাংলা ব্যাখ্যা

Algorithm সবসময় এমন neighbour state নেয় —

👉 যেটার **Evaluation Function value** বেশি।

মানে:

Better → Better → More Better।

Example (পাহাড় analogy)

তুমি পাহাড়ে উঠছো।

চারদিকে তাকাও।

যেদিক বেশি উঁচু মনে হয় সেখানে যাও।

ধাপে ধাপে ওপরে উঠছো।

এটাই Hill Climbing।

★ Evaluation Function

একটা function থাকে।

যেটা বলে দেয়:

👉 কোন state ভালো।

Example:

Score।

যার score বেশি।

সেটা ভালো।

★ Gradient Descent (Variation)

Slide:

Gradient descent search moves downhill.

Hill Climbing:

👉 Maximum value খোঁজে।

Gradient Descent:

👉 Minimum value খোঁজে।

Example:

Hill Climbing → Highest hill।

Gradient Descent → Lowest valley।

★ Loop Behaviour

Slide:

Continuously moves uphill and terminates when a peak is reached.

মানে:

Algorithm চলতেই থাকবে।

যতক্ষণ better neighbour পায়।

Stop করবে যখন:

👉 আর ভালো neighbour নাই।

মানে Peak এ পৌঁছেছে।

★ Immediate Neighbor দেখে

Slide:

Hill-climbing does not look ahead.

মানে:

Future চিন্তা করে না।

শুধু পাশের state দেখে।

Example:

Chess board এ শুধু এক move পরে কী হবে দেখে।

১০ move পরে কি হবে দেখে না।

★ Random Choice

Slide:

Chooses randomly among best successors.

যদি একাধিক equally ভালো state থাকে।

Example:

৩টা neighbour একই score।

তখন:

👉 Random একটা নেয়।

★ Greedy Local Search

Hill Climbing কে বলা হয়:

Greedy Local Search।

কারণ:

যেটা এখন best মনে হচ্ছে।

সেটাই নেয়।

Future risk দেখে না।

★ Memory Requirement

Slide:

Very simple strategy with low space requirement.

কারণ:

Search Tree রাখে না।

Slide:

Stores only state and evaluation.

মানে:

শুধু:

- Current State।
- Evaluation value।

এই দুইটা রাখে।

তাই:

👉 Space Complexity প্রায় $O(1)$ ।

★ 8 Queens Problem Example

এটা খুব famous example।

Chess Board এ ৪টা Queen বসাতে হবে।

যেন কেউ কাউকে attack না করে।

Successor Function

Slide:

Move a single queen in same column.

মানে:

একটা Queen একই column এ অন্য square এ সরানো যাবে।

Example:

Column 1 এর Queen।

Row 2 → Row 5 move।

Heuristic Function — $h(n)$

Slide:

Number of attacking queen pairs.

মানে:

কয়টা Queen একে অপরকে attack করছে।

Example:

5 pair attack করছে।

$h(n)=5$ ।

Goal:

👉 $h(n)=0$ ।

মানে:

কেউ কাউকে attack করছে না।

★ Hill Climbing Example

(a) State with $h = 17$

ধরি:

Current board এ 17টা conflict আছে।

প্রতি queen move করলে নতুন state পাওয়া যায়।

সব neighbour এর h calculate করা হয়।

Example:

Move করলে:

$h=15$

$h=12$

$h=9$

$h=20$

Best হলো:

👉 $h=9$ ।

Algorithm সেটাই নেয়।

এভাবে কমাতে থাকে।

$17 \rightarrow 9 \rightarrow 5 \rightarrow 2 \rightarrow 1 \rightarrow 0$ ।

Goal।

★ Local Minimum Problem

Slide:

Local minimum example.

ধরি:

Current state:

$h=1$ ।

Goal না।

কিন্তু সব neighbour:

$h=3$

$h=4$

$h=5$ ।

সব খারাপ।

Hill Climbing বলে:

Better neighbour নাই।

Stop।

কিন্তু Goal ছিল।

Algorithm miss করলো।

এটাই:

👉 Local Minimum।

★ Hill Climbing Problems

- ① Local Maximum।
 - ② Plateau (সব same value)।
 - ③ Ridge।
-

কারণ:

Future দেখে না।

★ Advantages

- ✓ খুব সহজ।
 - ✓ কম Memory লাগে।
 - ✓ Fast।
-

★ Disadvantages

- ✗ Complete না।
 - ✗ Optimal না।
 - ✗ Local minimum এ আটকে যায়।
-

★ Summary (Exam Ready)

Hill Climbing is a greedy local search algorithm that continuously moves toward a neighboring state with a better evaluation value. It considers only immediate neighbors and stops when no better successor is available. It requires very little memory because it stores only the current state and its evaluation. However, it may fail due to local minima or plateaus.

★ Comparison

Method	Key Idea	Advantage	Problem Solved
Stochastic HC	Random uphill selection	Avoid deterministic choice	Escapes tie situations in plateaus
First-Choice HC	Stop at first better neighbour	Fast	Reduces computation
Random-Restart HC	Multiple random starts	Avoid local maxima	Increases chance of global maximum